23. State Yönetimi & Veri Akışı: Kaosu Önlemek

Prop Drilling, Global State ve Asenkron Durum Yönetimi Rehberi
Bölüm 4 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Modern web uygulamalarında en çok bug nerede çıkar? Neden veritabanı veya sunucu değil, istemcideki 'State (Durum)' yönetimi başımızı ağrıtır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 State (Durum), uygulamanın belirli bir andaki hafızasıdır; kontrolsüz mutasyonlar kaosa yol açar. - 📌 Sunucu Durumu (Server State - React Query / SWR) ile İstemci Durumu (Client State - Zustand) birbirine karıştırılmamalıdır. - 📌 Tek Yönlü Veri Akışı (Unidirectional Data Flow), durum değişikliklerini öngörülebilir ve izlenebilir kılar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Büyük bir ailenin ortak bir buzdolabı olduğunu düşünün.

Eğer evdeki 5 kişi de buzdolabındaki sütü kafasına göre kafaya dikip içerse, kimse sütün bittiğinden haberdar olmaz. Anne kahve yapacakken süt kutusunun boş olduğunu görünce çığlık atar! İşte bu **kontrolsüz mutable state**'tir.

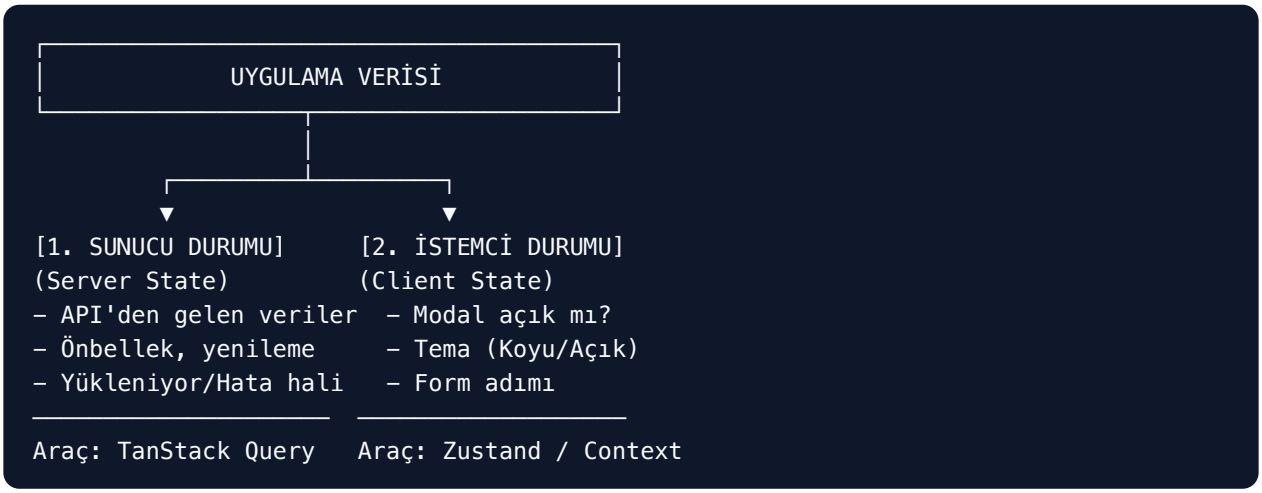
Peki buzdolabının kapağına bir liste koysanız: - Sütü alan kişi listeden bir çizgi çeker. - Son süt bittiğinde otomatik olarak sipariş listesine eklenir.

Artık herkes ne olduğunu bilir ve sürpriz yaşanmaz!

Yazılımda Zustand, Redux veya React Query işte o buzdolabı listesidir. Verinin kim tarafından, ne zaman ve nasıl değiştirildiğini kayıt altına alır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Modern State Ayrımı Matrisi:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Frontend Mimarisi ve React Uzmanı
Uygulamamızda hem sunucudan gelen ürün listesi var hem de kullanıcının sepet durumu var.

## GÖREV:
1. Neden ürün listesini `useEffect` ve `useState` ile yönetmenin kötü bir alışkanlık olduğunu açıkla.
2. Ürün listesini TanStack Query (React Query) kullanarak önbellekli ve otomatik yenilenen şekilde kodla.
3. Sepet durumunu (ekle/çıkar/toplam) hafif bir Zustand store'u olarak tasarla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü: useEffect ile veri çekmek (Race condition, önbellek yok, boilerplate dolu)
function ProductListNaive() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Bileşen unmount olursa memory leak ve race condition riski!
    fetch("/api/products")
      .then(res => res.json())
      .then(data => {
        setProducts(data);
        setLoading(false);
      });
  }, []); // Bağımlılıklar güncellenmediğinde bayat veri!
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// 1. İSTEMCİ DURUMU: Zustand ile Sepet Yönetimi (cartStore.ts)
import { create } from "zustand";

interface CartItem {
  id: string;
  name: string;
  price: number;
}

interface CartStore {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  clearCart: () => void;
  total: () => number;
}

export const useCartStore = create<CartStore>((set, get) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
  clearCart: () => set({ items: [] }),
  total: () => get().items.reduce((sum, item) => sum + item.price, 0)
}));

// 2. SUNUCU DURUMU: TanStack Query ile Ürün Çekme (products.hook.ts)
import { useQuery } from "@tanstack/react-query";

export function useProducts() {
  return useQuery({
    queryKey: ["products"],
    queryFn: async () => {
      const res = await fetch("/api/products");
      if (!res.ok) throw new Error("Ürünler yüklenemedi.");
      return (await res.json()) as CartItem[];
    },
    staleTime: 1000 * 60 * 5 // 5 dakika boyunca önbellekten anında sun
  });
}
```

💡 **Kod Analizi:** TanStack Query; otomatik önbellekleme (caching), arka planda yenileme (revalidation) ve yüklenme durumlarını tek satırda çözerken, Zustand yerel sepet hafızasını prop drilling olmadan yönetir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Sunucudan gelen her API verisini Redux veya global state'e kopyalayıp senkronizasyon kaybetmek.

  - **Çözüm:** Sunucu verisi için TanStack Query kullanın; Redux/Zustand'ı sadece gerçek UI durumları için saklayın.
- **Tuzak:** React state'ini doğrudan mutasyona uğratmak (Örn: `state.items.push(item)`).
  - **Çözüm:** State daima immutable (değişmez) güncellenmelidir (`[...state.items, item]`).
- **Tuzak:** Prop drilling yaparak bir veriyi 8 alt bileşene elle aktarmak.
  - **Çözüm:** Derin hiyerarşilerde Zustand veya React Context kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 23: State Yönetimi - TanStack Query ve Zustand ile Düzen*

- 🖥️ **Slayt 1:** Frontend bug'larının %70'i neden State yönetiminden kaynaklanır?
- 🖥️ **Slayt 2:** Server State (Sunucu) vs Client State (İstemci) ayrımı.
- 🖥️ **Slayt 3:** useEffect veri çekme tuzağından TanStack Query'ye geçiş.
- 🖥️ **Slayt 4:** Zustand ile 10 satırda global sepet yönetimi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
React uygulamasında iki bileşen açın. Biri sepete ürün eklesin, diğeri header'da sepet adedini göstersin. Zustand store'u ile prop aktarmadan anında güncellendiğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `useEffect` ile veri çektiğiniz bir yeri bulun ve TanStack Query `useQuery` kullanarak yeniden yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: React uygulamalarında sunucudan (API) gelen veriyi yönetmek için modern standart hangi araçtır? - A) Sadece `useState` ve `useEffect` - B) TanStack Query (React Query) veya SWR gibi Sunucu Durumu (Server State) kütüphaneleri - C) jQuery - D) CSS Animations > **Doğru Yanıt:** B > **Açıklama:** TanStack Query önbellekleme, deduplication, retry ve otomatik senkronizasyon özelliklerini otomatik sunar.

Soru 2: React state'inde `state.users.push(newUser)` yazmak neden tehlikelidir? - A) Kodun derlenmemesi - B) Dizinin referansı değişmediği için React'in değişikliği fark edememesi ve arayüzü yeniden render etmemesi (Bailout) - C) Bilgisayarın kilitlenmesi - D) Sadece Python'da hata vermesi > **Doğru Yanıt:** B > **Açıklama:** React sığ karşılaştırma (shallow equality) yapar. Dizi doğrudan mutasyona uğradığında referans değişmediği için render tetiklenmez.
